



SUBJECT : PROGRAMMING IN JAVA

SUBJECT CODE : CSI26W1

LEARNING UNIT: 5 (EXCEPTION HANDLING)

PROGRAMMING EXERCICES

Task 1:

The `Double.parseDouble()` method requires a `String` argument, but it fails if the `String` cannot be converted to a floating-point number. Write an application in which you try accepting a double input from a user and catch a `NumberFormatException` if one is thrown. The catch block forces the number to 0 and displays an appropriate error message. Following the catch block, display the number.

Save the file as **`ParseDouble.java`**.

Task 2:

Write an application that prompts the user to enter an integer and displays its square root. Accept the user's entry as a `String` using the `nextLine()` method. Then try to use the `Integer.parseInt()` method to convert the `String` to an integer. If the conversion throws an `Exception`, display an error message, and set the number to 0. Then, if the entered number is less than 0, throw an `ArithmeticException` to a catch block that explains you cannot take the square root of a negative number. If the entered number is not less than 0, take the square root of the number using the `Math.sqrt()` method, and display the result.

Save the file as **`SquareRootException.java`**.

Task 3:

Write a program that prompts the user to enter an integer that represents an array size. Java generates a `NumberFormatException` if the user attempts to enter a non-integer value using `nextInt()`. Handle this exception by displaying an appropriate error message. If the size entered is an integer, create an array of type `double` using the integer entered as the size. Java generates a `NegativeArraySizeException` if you attempt to create an array with a negative size; handle this exception by setting the array size to a default value of five. If the array is created successfully, use exception-handling techniques to ensure that

each entered array value is a double. Then calculate each element's distance from the average.

Save the file as **DistanceFromAverageExceptionHandling.java**.

Task 4:

- a) Create a `UsedCarException` class that extends `Exception`; its constructor receives a value for a vehicle identification number (VIN) that is passed to the parent constructor so it can be used in a `getMessage()` call.

Save the class as **UsedCarException.java**.

- b) Create a `UsedCar` class with fields for a VIN, make, year, mileage, and price. The `UsedCar` constructor throws a `UsedCarException` when the VIN is not four digits; when the make is not Ford, Honda, Toyota, Chrysler, or Other; when the year is not between 1997 and 2024 inclusive; or either the mileage or price is negative.

Save the class as **UsedCar.java**.

- c) Write an application that establishes an array of at least seven `UsedCar` objects and prompts the user for values. Include two catch blocks. One catches any data entry Exceptions and displays a message. The other catches `UsedCarExceptions` that are thrown from the `UsedCar` constructor. When data entry is complete, display a list of only the `UsedCar` objects that were constructed successfully.

Save the file as **ThrowUsedCarException.java**.

~~~~~